Manual de usuario — PromptZal

Proyecto 2 — Laboratorio de Lenguajes Formales y de Programación

Este manual explica cómo usar la aplicación PromptZal para compilar un archivo `.pz`, revisar errores y obtener el prompt final.

1. ¿Qué hace PromptZal?

PromptZal toma un programa escrito en el lenguaje `.pz` y genera un prompt en texto listo para usarse con una herramienta de IA.

El programa puede contener:

- Directivas: `@modelo`, `@rol`, `@formato`.
- Agentes con contexto, variables y comandos.
- Ejecuciones con `EJECUTAR`.
- Resultados solicitados con `EXPORTAR`.

La aplicación muestra el prompt generado, los errores de la compilación y reportes HTML.

2. Requisitos

- Java 21.
- Maven para compilar desde consola.
- Un navegador para abrir los reportes HTML.

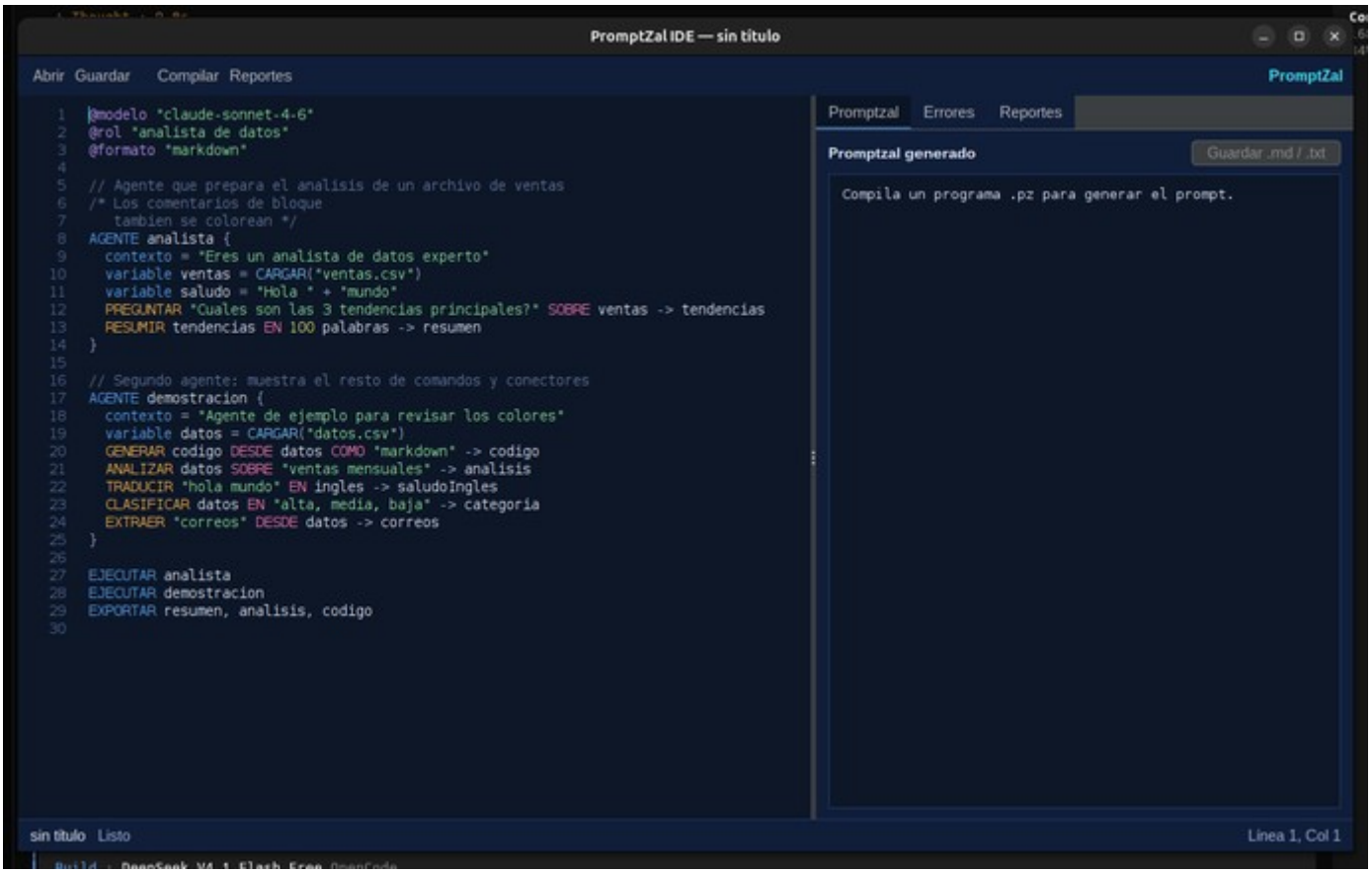
3. Ejecutar la aplicación

Desde la carpeta del proyecto:

```
mvn clean package
java -jar target/PromptZal_3-1.0-SNAPSHOT.jar
```

También se puede abrir el proyecto en NetBeans y ejecutar la clase principal.

4. Ventana principal



La ventana se divide en:

- **Barra superior:** Abrir, Guardar, Compilar y Reportes.
- **Editor:** zona donde se escribe o abre el programa `.pz`.
- **Panel derecho:** pestañas Promptzal, Errores y Reportes.
- **Barra inferior:** nombre del archivo, estado y posición del cursor.

Al iniciar, la aplicación carga un programa de ejemplo para mostrar el resaltado de sintaxis.

5. Abrir un archivo `.pz`

1. Pulsar **Abrir**.
2. Seleccionar un archivo con extensión `.pz`.
3. El contenido aparece en el editor.

Si hay cambios sin guardar, la aplicación pregunta si se desean descartar antes de abrir otro archivo.

6. Guardar un archivo `.pz`

1. Pulsar **Guardar**.
2. Si el archivo no tiene ruta, elegir dónde guardarlo.
3. La aplicación agrega automáticamente la extensión `.pz` si falta.

Si el archivo ya existe, se pide confirmación antes de sobrescribirlo.

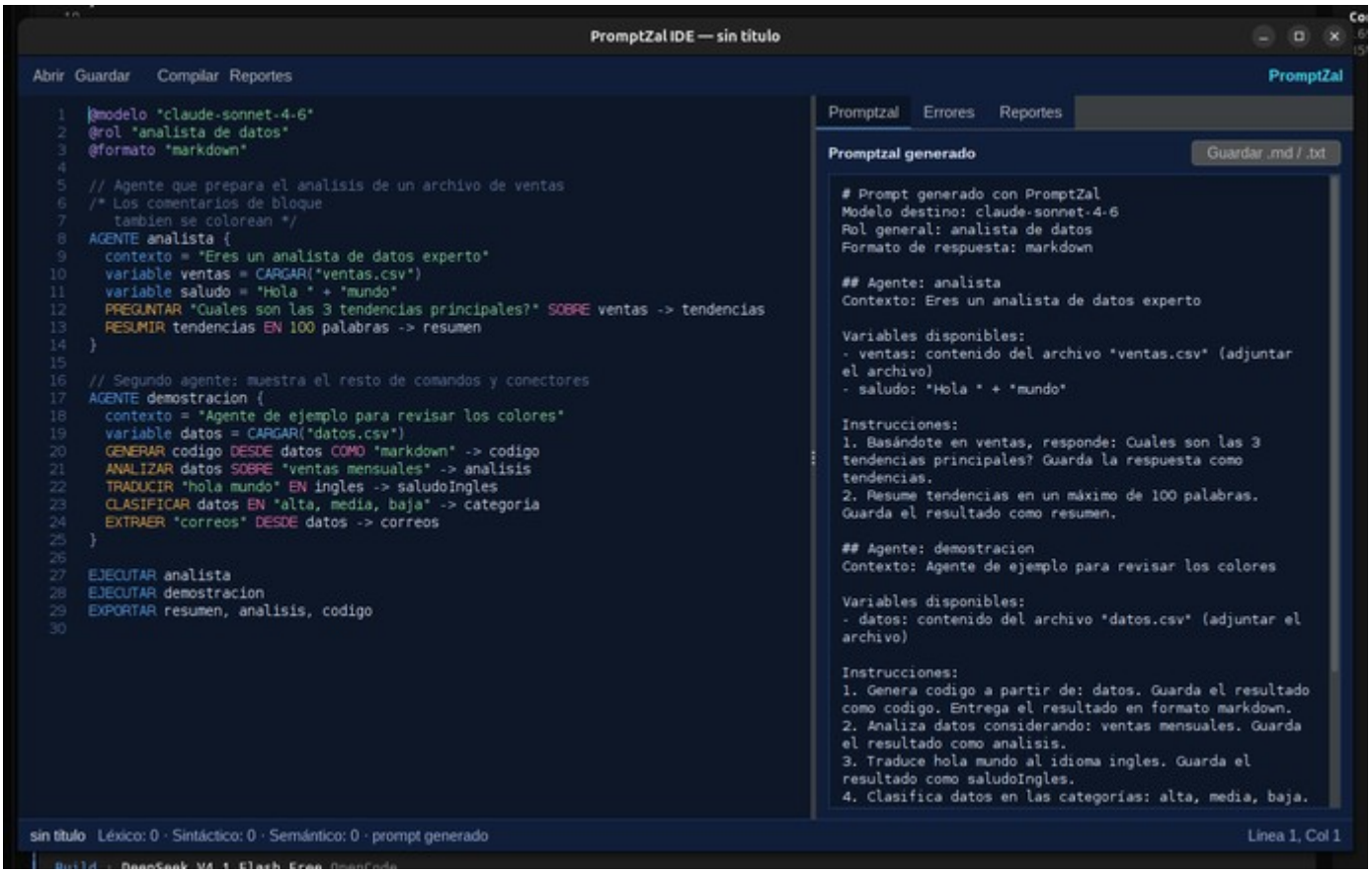
7. Compilar

1. Escribir o abrir un programa.
2. Pulsar **Compilar**.
3. La aplicación ejecuta las fases léxica, sintáctica, semántica y de generación.
4. Al terminar:

- Si no hay errores, se muestra la pestaña **Promptzal**.
- Si hay errores, se muestra la pestaña **Errores**.
- La barra inferior indica cuántos errores léxicos, sintácticos y semánticos se encontraron.

La compilación se ejecuta en segundo plano, por lo que la ventana no se congela.

8. Prompt generado



La pestaña **Promptzal** muestra el prompt final.

Incluye:

- Modelo, rol y formato, si fueron declarados.
- Agentes ejecutados.
- Contexto de cada agente.
- Variables y su origen.
- Instrucciones generadas a partir de los comandos.
- Resultados solicitados en `EXPORTAR`.

Guardar el prompt

1. Pulsar **Guardar .md / .txt**.
2. Elegir la ubicación.
3. Escribir el nombre.
4. La aplicación agrega `.md` si no se indica `.md` o `.txt`.

El archivo se guarda en UTF-8.

9. Panel de errores

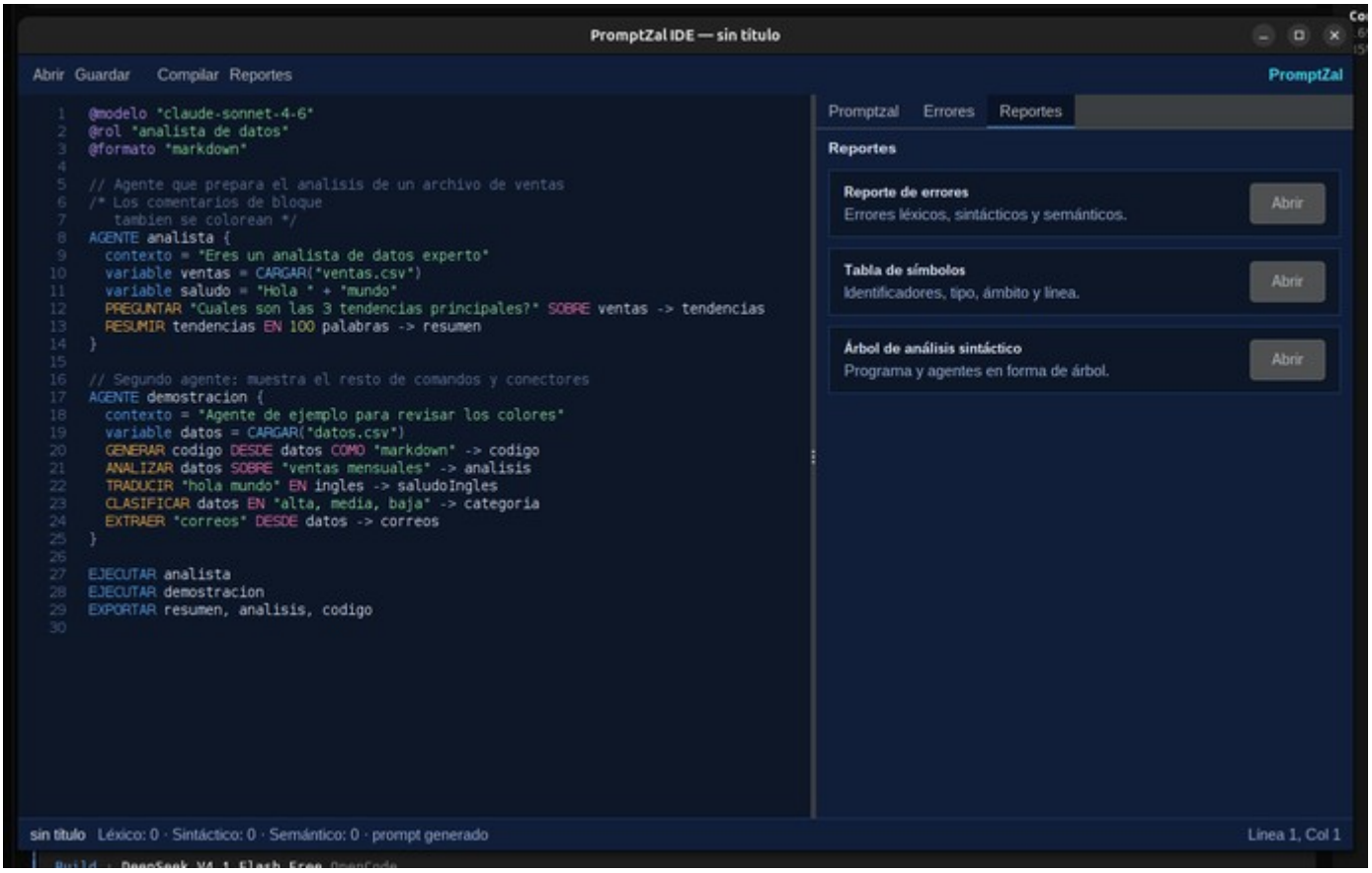
La pestaña **Errores** muestra una tabla con:

- Tipo: léxico, sintáctico o semántico.
- Descripción.
- Fila.
- Columna.
- Token encontrado.
- Token esperado.

Si se hace doble clic sobre una fila, el editor salta a la línea del error.

Los errores sintácticos muestran el token encontrado y el esperado. Los errores léxicos y semánticos muestran la descripción y la ubicación.

10. Reportes



La pestaña **Reportes** tiene tres botones:

- **Reporte de errores**

- **Tabla de símbolos**
- **Árbol de análisis sintáctico**

Cada botón genera un archivo HTML y lo abre en el navegador.

Si todavía no se ha compilado, la aplicación muestra un mensaje indicando que primero se debe compilar.

Reporte de errores

Muestra todos los errores de la última compilación. Si no hay errores, indica explícitamente que el programa es válido.

Tabla de símbolos

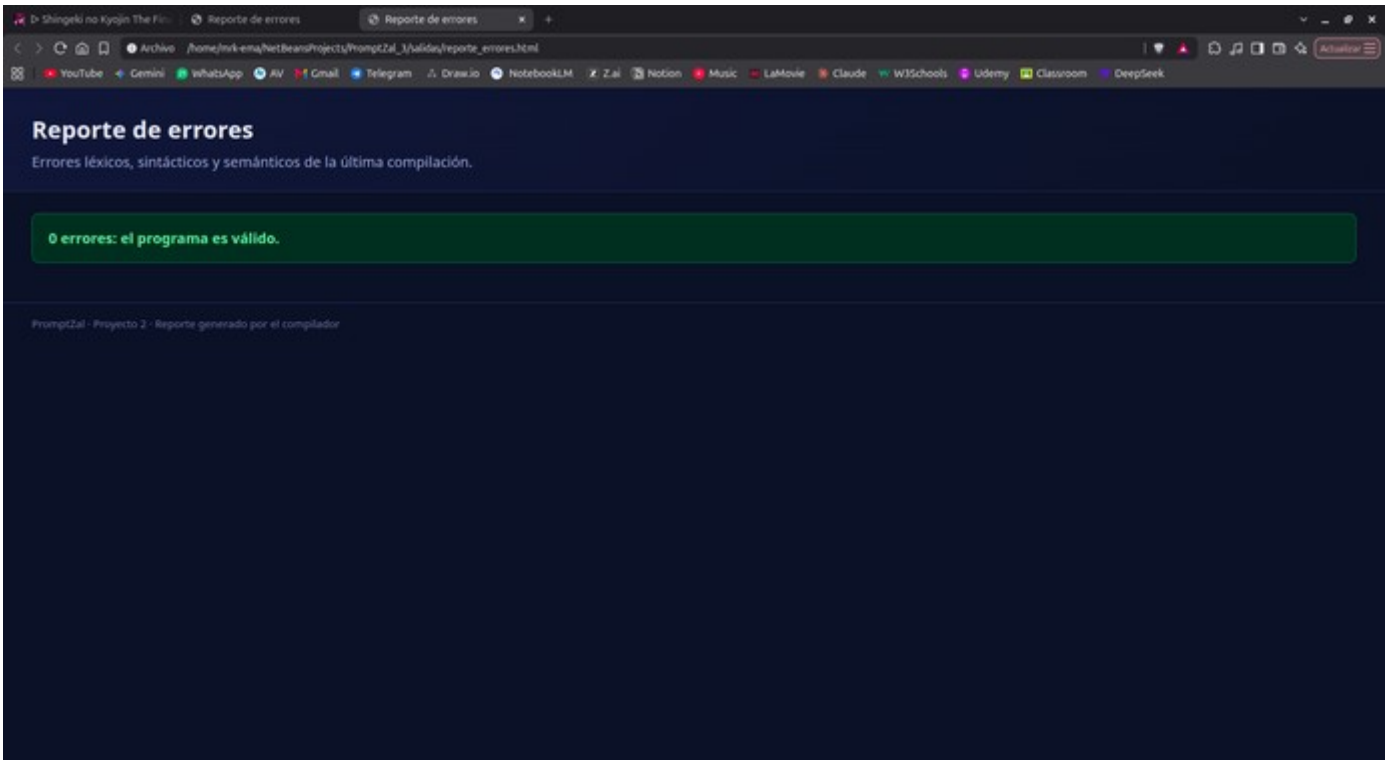
Muestra agentes, variables declaradas y resultados de comandos con:

- Nombre.
- Tipo.
- Ámbito.
- Línea.

Árbol de análisis sintáctico

Muestra el AST en forma de árbol con sangrías. Incluye directivas, agentes, contexto, variables, comandos, conectores, ejecuciones y exportaciones.

11. Reporte abierto en el navegador



Los reportes son archivos HTML autocontenidos. Se pueden abrir con cualquier navegador y no necesitan un servidor.

Se guardan en la carpeta `salidas/` del directorio desde donde se ejecuta la aplicación:

```
salidas/
├── reporte_errores.html
├── reporte_simbolos.html
└── reporte_ast.html
```

Cada vez que se genera un reporte, se sobrescribe el anterior.

12. Ejemplo de uso

1. Abrir `pruebas/referencia.pz`.
2. Pulsar **Compilar**.
3. Revisar el prompt en **Promptzal**.
4. Guardar el prompt con otro nombre, por ejemplo `prompt_referencia.md`.
5. Entrar a **Reportes**.
6. Abrir el reporte de errores: debe indicar que no hay errores.
7. Abrir la tabla de símbolos y el árbol sintáctico.

Para probar errores con el ejemplo de entrega:

1. Abrir `entrega/ejemplo.pz`.
2. Compilar.
3. Revisar la pestaña **Errores**: hay errores léxicos, sintácticos y semánticos.
4. Abrir el reporte de errores.

El prompt se genera igual desde el agente `analista`, que es válido. Los errores están en un agente que no se ejecuta.

También se puede probar con:

- `pruebas/lexico_errores.pz`
- `pruebas/semanticos.pz`
- `pruebas/ambitos.pz`
- `pruebas/declaracion_tardia.pz`
- `pruebas/excepciones.pz`
- `pruebas/exportar_inexistente.pz`

13. Archivos de entrega

La carpeta `entrega/` contiene un único ejemplo completo:

```
entrega/
├── ejemplo.pz
├── prompt_ejemplo.md
└── reportes/
    ├── reporte_errores.html
    ├── reporte_simbolos.html
    └── reporte_ast.html
```

- `ejemplo.pz`: programa de entrada. Tiene errores intencionales en el agente `errores`, que no se ejecuta; el agente `analista` es válido y genera el prompt.
- `prompt_ejemplo.md`: prompt generado desde `entrega/ejemplo.pz`.
- `reportes/reporte_errores.html`: reporte con errores léxicos, sintácticos y semánticos.
- `reportes/reporte_simbolos.html`: tabla de símbolos de la compilación.
- `reportes/reporte_ast.html`: AST construido por el parser.

14. Problemas comunes

No se abre el reporte

El reporte se genera igual. La aplicación muestra la ruta del archivo. Se puede abrir manualmente desde el explorador de archivos.

El archivo no se guarda como `.pz`

La aplicación corrige la extensión automáticamente. Si se escribe `programa.txt`, se guarda como `programa.pz`.

Hay errores y no aparece el prompt

El prompt se genera igual, pero la aplicación muestra primero la pestaña de errores. Se puede revisar el prompt en la pestaña **Promptzal**.

El prompt está incompleto

Puede deberse a errores sintácticos o semánticos. Revisar la pestaña **Errores** y corregir el programa.

15. Resumen rápido

1. Abrir un `.pz`.
2. Compilar.
3. Revisar errores.
4. Revisar y guardar el prompt.
5. Abrir los reportes HTML.